CHAPTER 2

CONCEPTS OF OBJECT ORIENTED PROGRAMMING

CHAPTER ESSENCE AT A GLANCE:
Master the paradigm shift from procedural to object-oriented programming, data encapsulation, inheritance hierarchies, and dynamic polymorphism.

4 CORE PRINCIPLES TO UNDERSTAND FIRST

• **Paradigm Shift and Modularity:** Object-oriented programming shifts focus from sequential procedural steps to autonomous software entities called objects, unifying state and behavioral operations into cohesive modular classes.

• **Encapsulation and Information Hiding:** Data encapsulation wraps fields and methods inside a protective class boundary, restricting outside interference by exposing only controlled public interfaces while keeping critical implementation details private.

• **Inheritance and Code Reusability:** Derivation allows specialized child classes to inherit attributes and methods from existing base classes, eliminating redundant code while establishing a clean hierarchical classification.

• **Polymorphism and Dynamic Dispatch:** Polymorphism enables a single unified interface to control distinct underlying operations, accomplished at compile time via function and operator overloading, or at run time through virtual method binding.

ESSENTIAL ARCHITECTURE & CORE SYNTAX CARDS

$$\text{Class} = \text{Data Members (State)} + \text{Member Functions (Behavior)}$$

Core object-oriented encapsulation definition

$$\text{public} > \text{protected} > \text{private}$$

Access specifier visibility scope hierarchy

$$\text{Construction: Base} \rightarrow \text{Derived} \quad | \quad \text{Destruction: Derived} \rightarrow \text{Base}$$

Constructor and destructor execution sequence in inheritance

$$\text{Signature} = \text{Function Name} + \text{Parameter Types} + \text{Parameter Count}$$

Compile-time polymorphism function overloading signature

CRUCIAL EXAM PRECAUTIONS & VALUATION RUBRIC SECRETS

• **Alert:** Private members cannot be accessed directly via dot operator from outside the class scope.

• **Alert:** Constructors must never have a return type, not even void; defining one converts it to a regular member function.

• **Alert:** Destructors are invoked automatically in reverse order of constructor execution upon object destruction.

HIGH-YIELD CENTUM QUESTIONS & STEP-BY-STEP EXPLANATIONS

1. Explain the paradigm shift from Procedural Programming to Object Oriented Programming. [Score: 2 Marks]

Answer & Key Valuation Steps:

• **Step 1 (Procedural vs Object-Oriented):** Procedural programming (C, Pascal) focuses on functions/procedures and sequence of actions; data moves freely and is vulnerable. OOP (C++, Java) models real-world entities as objects; binds data and functions together into a single unit (class), hiding data and preventing unauthorized access. **[2 Marks]**

• **Key Exam Result:** Procedural (function-centric) vs OOP (data-centric, secure)

$$\text{Class} = \text{Data Members (State)} + \text{Member Functions (Behavior)}$$

Core object-oriented encapsulation definition

2. Explain the four fundamental pillars of OOP: Encapsulation, Data Abstraction, Inheritance, and Polymorphism. [Score: 4 Marks]

Answer & Key Valuation Steps:

• **Step 1 (Four Pillars of OOP):** 1. Encapsulation: Wrapping up data and functions into a single unit (class) to prevent direct outside interference. 2. Data Abstraction: Representing essential features of an object without including background details or implementation complexity. 3. Inheritance: Mechanism by which a derived class inherits properties and behaviors from a base class (code reusability). 4. Polymorphism: Ability of a message or function to be displayed or executed in multiple forms (function overloading, operator overloading, virtual functions). **[4 Marks]**

• **Key Exam Result:** Encapsulation (wrapping), Abstraction (hiding detail), Inheritance (reuse), Polymorphism (multiple forms)

$$\text{Class} = \text{Data Members (State)} + \text{Member Functions (Behavior)}$$

Core object-oriented encapsulation definition

3. Explain class and object in C++. Explain the private, protected, and public access specifiers. [Score: 3 Marks]

Answer & Key Valuation Steps:

• **Step 1 (Class & Object):** Class is a user-defined blueprint or template; Object is a runtime instance of a class. **[1 Mark]**

• **Step 2 (Access Specifiers):** private: accessible only by member functions of the same class (default). protected: accessible within the same class and its derived (inherited) classes. public: accessible from anywhere in the program where the object is visible. **[2 Marks]**

• **Key Exam Result:** private (class only) | protected (class + derived) | public (universal)

$$\text{Class} = \text{Data Members (State)} + \text{Member Functions (Behavior)}$$

Core object-oriented encapsulation definition

4. Write a C++ program with a class 'Rectangle' to compute area and perimeter. [Score: 3 Marks]

Answer & Key Valuation Steps:

• **Step 1 (Class Code):** class Rectangle {n private: float length, breadth;n public:n void getData() { cout << 'Enter length and breadth: '; cin >> length >> breadth; }n float area() { return length * breadth; }n float perimeter() { return 2 * (length + breadth); }n};nint main() { Rectangle r; r.getData(); cout << 'Area: ' << r.area(); return 0; } **[3 Marks]**

• **Key Exam Result:** Class Rectangle with private members, public member functions

$$\text{Class} = \text{Data Members (State)} + \text{Member Functions (Behavior)}$$

Core object-oriented encapsulation definition

5. What is a constructor? Explain Default, Parameterized, and Copy constructors in C++. [Score: 3 Marks]

Answer & Key Valuation Steps:

• **Step 1 (Constructor):** A special member function having the same name as the class used to automatically initialize objects upon creation. Has NO return type (not even void). Automatically invoked. **[1.5 Marks]**

• **Step 2 (Types):** 1. Default constructor (takes no arguments). 2. Parameterized constructor (accepts arguments). 3. Copy constructor (initializes object using another object of same class: ClassName(const ClassName &obj)). **[1.5 Marks]**

• **Key Exam Result:** Default, Parameterized, and Copy constructors

$$\text{Construction: Base} \rightarrow \text{Derived} \quad | \quad \text{Destruction: Derived} \rightarrow \text{Base}$$

Constructor and destructor execution sequence in inheritance

6. What is a destructor? Explain its role, syntax, and characteristics in C++ memory management. [Score: 2 Marks]

Answer & Key Valuation Steps:

• **Step 1 (Destructor Concept):** A special member function having the same name as the class preceded by a tilde (~). Automatically executed when an object goes out of scope or is deleted. **[1 Mark]**

• **Step 2 (Role in Memory Deallocation):** Deallocates dynamically allocated heap memory (calling delete on member pointers) and releases system resources (closing files/connections), preventing memory leaks. Cannot be overloaded and takes no parameters. **[1 Mark]**

• **Key Exam Result:** ~ClassName() automatically releases resources and heap memory

$$\text{Construction: Base} \rightarrow \text{Derived} \quad | \quad \text{Destruction: Derived} \rightarrow \text{Base}$$

Constructor and destructor execution sequence in inheritance

7. Explain the concept of Inheritance in C++. Describe the five types of inheritance with syntax. [Score: 4 Marks]

Answer & Key Valuation Steps:

• **Step 1 (Concept):** Mechanism of deriving a new class (derived or child class) from an existing class (base or parent class), promoting code reusability. **[1.5 Marks]**

• **Step 2 (Five Types with Syntax):** 1. Single (B : public A). 2. Multilevel (C : public B : public A). 3. Multiple (C : public A, public B). 4. Hierarchical (B : public A and C : public A). 5. Hybrid (combination of two or more types). **[2.5 Marks]**

• **Key Exam Result:** Single, Multilevel, Multiple, Hierarchical, Hybrid inheritance

$$\text{Reusability Ratio} = \frac{\text{Inherited Code Lines}}{\text{Total Application Lines}}$$

Inheritance software reuse efficiency metric